Simulation Study for Pseudo-value Regression

FengQi (Cynthia) Song

2026-03-29

Contents

---

1 Objective - evaluate the behavior of pseudo-value regression for recurrent event outcomes under the null and alternative settings.

1.1 Sanity check

```
library(MCC)

scenarios <- data.frame(
  scenario = c("baseline", "high_censor", "high_death", "high_both"),
  lambda_c = c(0.25, 0.40, 0.25, 0.40),
  lambda_d = c(0.25, 0.25, 0.40, 0.40)
)

lambda_c <- scenarios$lambda_c[1]
lambda_d <- scenarios$lambda_d[1]

source("01_generate_data.R")
t_eval <- 5
pseudo <- MCC::GenPseudo(data = dat, tau = t_eval)

str(pseudo)
```

```
## 'data.frame':    400 obs. of  3 variables:
##  $ idx   : num  1 2 3 4 5 6 7 8 9 10 ...
##  $ psi   : num  0.1959 4.374 -0.5288 0.0767 0.142 ...
##  $ pseudo: num  2.06 6.24 1.33 1.94 2 ...
```

```
head(pseudo)
```

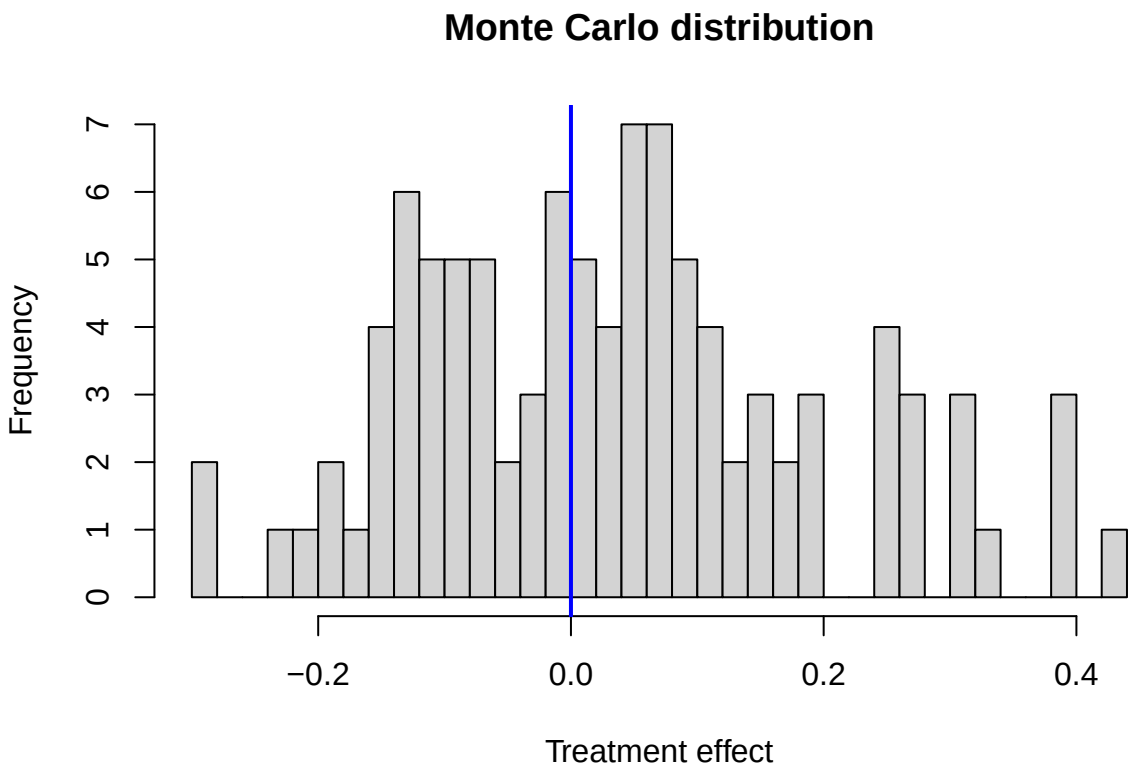
```
##   idx         psi     pseudo
## 1   1  0.19589729  2.0583226
## 2   2  4.37396253  6.2363878
## 3   3 -0.52881330  1.3336120
## 4   4  0.07671525  1.9391406
## 5   5  0.14201115  2.0044365
## 6   6 -1.97494103 -0.1125157
```

```
AX <- unique(dat[, c("idx", "A", "X")])
merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

tapply(merged$pseudo, merged$A, mean)
```

```
##        0        1
## 1.992021 1.732830
```

1.2 Monte Carlo validation

```
library(MCC)
lambda_c <- 0.25
lambda_d <- 0.25

source("01_generate_data.R")

set.seed(123)

B <- 100
delta_hat <- numeric(B)

for (b in 1:B) {
  source("01_generate_data.R")
  pseudo <- MCC::GenPseudo(data = dat, tau = 5)

  AX <- unique(dat[, c("idx", "A")])
  merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

  delta_hat[b] <- mean(merged$pseudo[merged$A == 1]) -
                  mean(merged$pseudo[merged$A == 0])
}

hist(delta_hat, breaks = 30,
     main = "Monte Carlo distribution",
     xlab = "Treatment effect")
abline(v = 0, col = "blue", lwd = 2)
```



This distribution is centered at zero, suggesting no bias

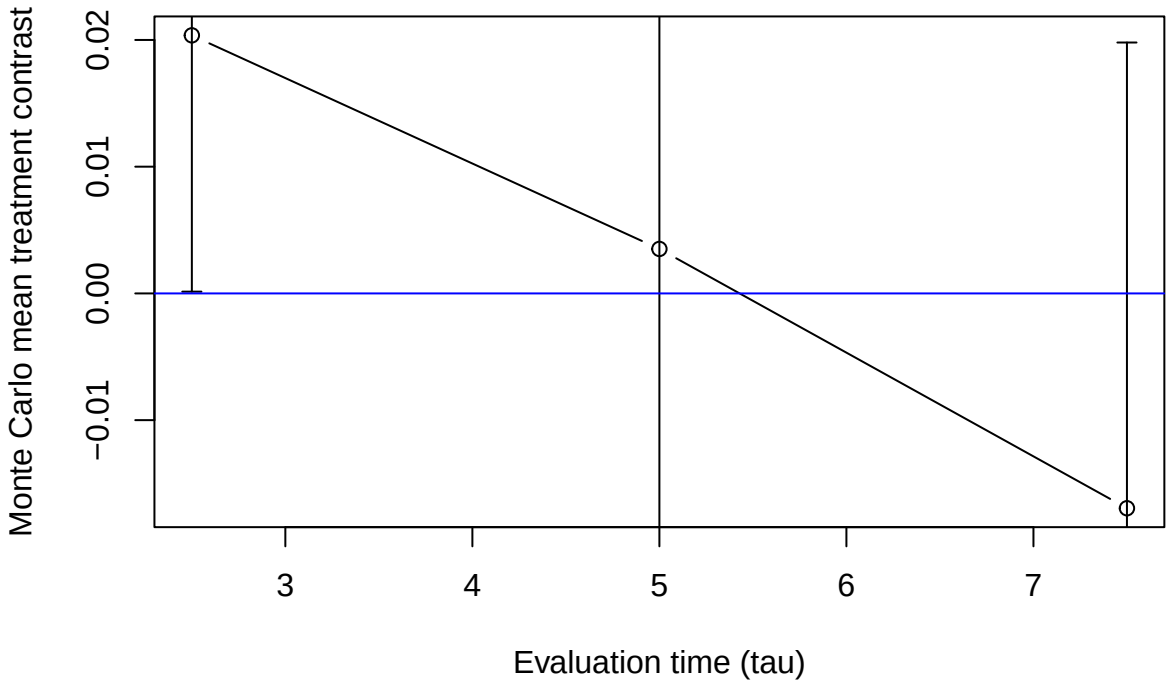
1.3 Sensitivity analysis(different evaluation time)

```
set.seed(123)

lambda_c <- 0.25
lambda_d <- 0.25

B <- 100
t_list <- c(2.5, 5, 7.5)

out_time <- data.frame(
  t_eval = t_list,
  mean_diff = NA_real_,
  sd_diff = NA_real_,
  se_diff = NA_real_,
  lower = NA_real_,
  upper = NA_real_
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
```

```r
  delta_tau <- numeric(B)

  for (b in 1:B) {

    source("01_generate_data.R")

    pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    merged_j <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

    delta_tau[b] <- mean(merged_j$pseudo[merged_j$A == 1]) -
                    mean(merged_j$pseudo[merged_j$A == 0])
  }

  out_time$mean_diff[j] <- mean(delta_tau)
  out_time$sd_diff[j]   <- sd(delta_tau)
  out_time$se_diff[j]   <- out_time$sd_diff[j] / sqrt(B)
  out_time$lower[j]     <- out_time$mean_diff[j] - 1.96 * out_time$se_diff[j]
  out_time$upper[j]     <- out_time$mean_diff[j] + 1.96 * out_time$se_diff[j]
}

out_time
```

```
##   t_eval    mean_diff   sd_diff    se_diff         lower      upper
## 1    2.5  0.020363700 0.1031788 0.01031788  0.0001406547 0.04058674
## 2    5.0  0.003509655 0.1425559 0.01425559 -0.0244313082 0.03145062
## 3    7.5 -0.016951416 0.1874941 0.01874941 -0.0537002574 0.01979742
```

```r
plot(out_time$t_eval, out_time$mean_diff, type = "b",
     xlab = "Evaluation time (tau)",
     ylab = "Monte Carlo mean treatment contrast")

arrows(
  x0 = out_time$t_eval,
  y0 = out_time$lower,
  x1 = out_time$t_eval,
  y1 = out_time$upper,
  angle = 90,
  code = 3,
  length = 0.05
)

abline(h = 0, col = "blue")
```



1.4 Pseudo-value Regression Model

```r
# Single-run regression across tau

t_list <- c(2.5, 5, 7.5)
AX <- unique(dat[, c("idx", "A")])

reg_out <- data.frame(
  t_eval = t_list,
  beta_A = NA,
  p_A = NA
)

for (j in seq_along(t_list)) {
  t_eval <- t_list[j]
  pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
  reg_dat <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

  fit <- lm(pseudo ~ A, data = reg_dat)
  s <- summary(fit)$coefficients

  reg_out$beta_A[j] <- s["A", "Estimate"]
  reg_out$p_A[j] <- s["A", "Pr(>|t|)"]
}
```

```
reg_out
```

```
##   t_eval      beta_A       p_A
## 1    2.5 -0.02916916 0.7706384
## 2    5.0 -0.03547660 0.8143822
## 3    7.5 -0.11427619 0.5501162
```

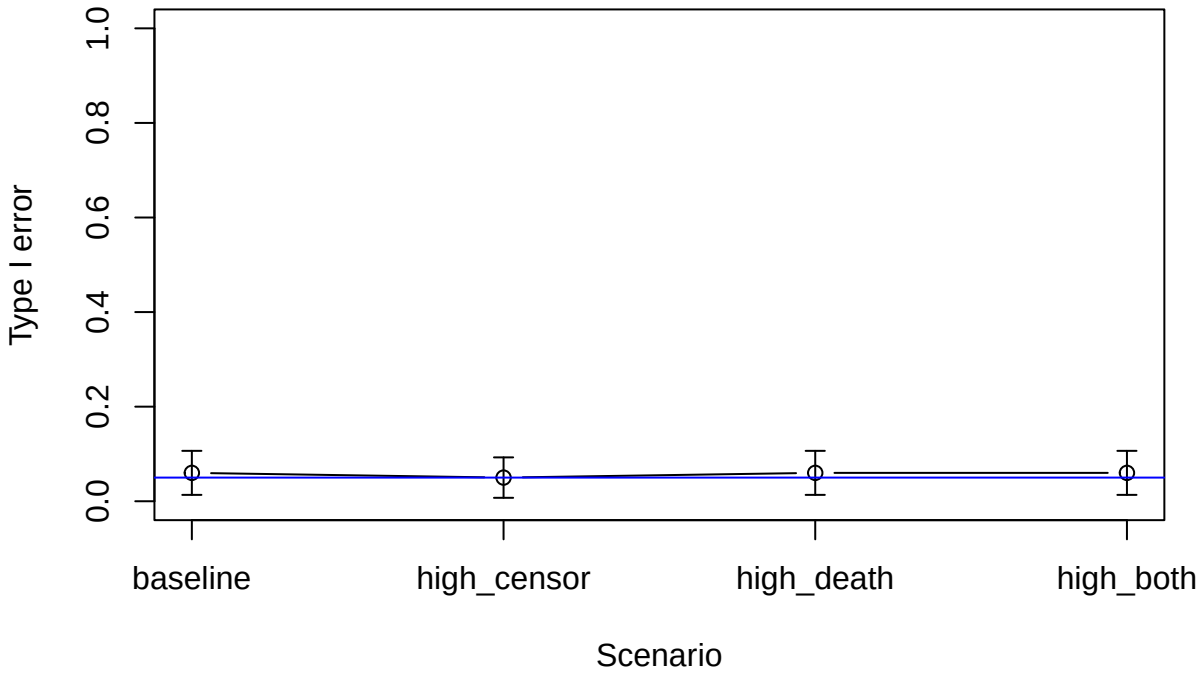
1.5 Type I error

```
set.seed(123)

B <- 100
alpha <- 0.05
t_eval <- 2.5
alphaA <- 0

type1_result <- data.frame(
  scenario = scenarios$scenario,
  lambda_c = scenarios$lambda_c,
  lambda_d = scenarios$lambda_d,
  reject_rate = NA_real_,
  mean_beta = NA_real_,
  se_reject = NA_real_,
  lower = NA_real_,
  upper = NA_real_
)

for (j in seq_len(nrow(scenarios))) {

  lambda_c <- scenarios$lambda_c[j]
  lambda_d <- scenarios$lambda_d[j]

  reject <- logical(B)
  beta <- numeric(B)

  for (b in 1:B) {

    source("01_generate_data.R")

    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)

    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)"] < alpha
  }
```

```
  type1_result$reject_rate[j] <- mean(reject)
  type1_result$mean_beta[j] <- mean(beta)

  type1_result$se_reject[j] <- sqrt(type1_result$reject_rate[j] *
                                      (1 - type1_result$reject_rate[j]) / B)

  type1_result$lower[j] <- max(0,
                               type1_result$reject_rate[j] -
                                 1.96 * type1_result$se_reject[j])

  type1_result$upper[j] <- min(1,
                               type1_result$reject_rate[j] +
                                 1.96 * type1_result$se_reject[j])
}

type1_result
```

```
##      scenario lambda_c lambda_d reject_rate    mean_beta  se_reject      lower
## 1    baseline     0.25     0.25        0.06  0.020363700 0.02374868 0.01345258
## 2 high_censor     0.40     0.25        0.05  0.003391898 0.02179449 0.00728279
## 3  high_death     0.25     0.40        0.06 -0.002011187 0.02374868 0.01345258
## 4   high_both     0.40     0.40        0.06  0.006910397 0.02374868 0.01345258
##        upper
## 1 0.10654742
## 2 0.09271721
## 3 0.10654742
## 4 0.10654742
```

```
plot(seq_len(nrow(type1_result)), type1_result$reject_rate, type = "b",
     xaxt = "n",
     ylim = c(0, 1),
     xlab = "Scenario",
     ylab = "Type I error")

axis(1, at = seq_len(nrow(type1_result)), labels = type1_result$scenario)

arrows(
  x0 = seq_len(nrow(type1_result)),
  y0 = type1_result$lower,
  x1 = seq_len(nrow(type1_result)),
  y1 = type1_result$upper,
  angle = 90,
  code = 3,
  length = 0.05
)

abline(h = 0.05, col = "blue")
```



1.6 Power

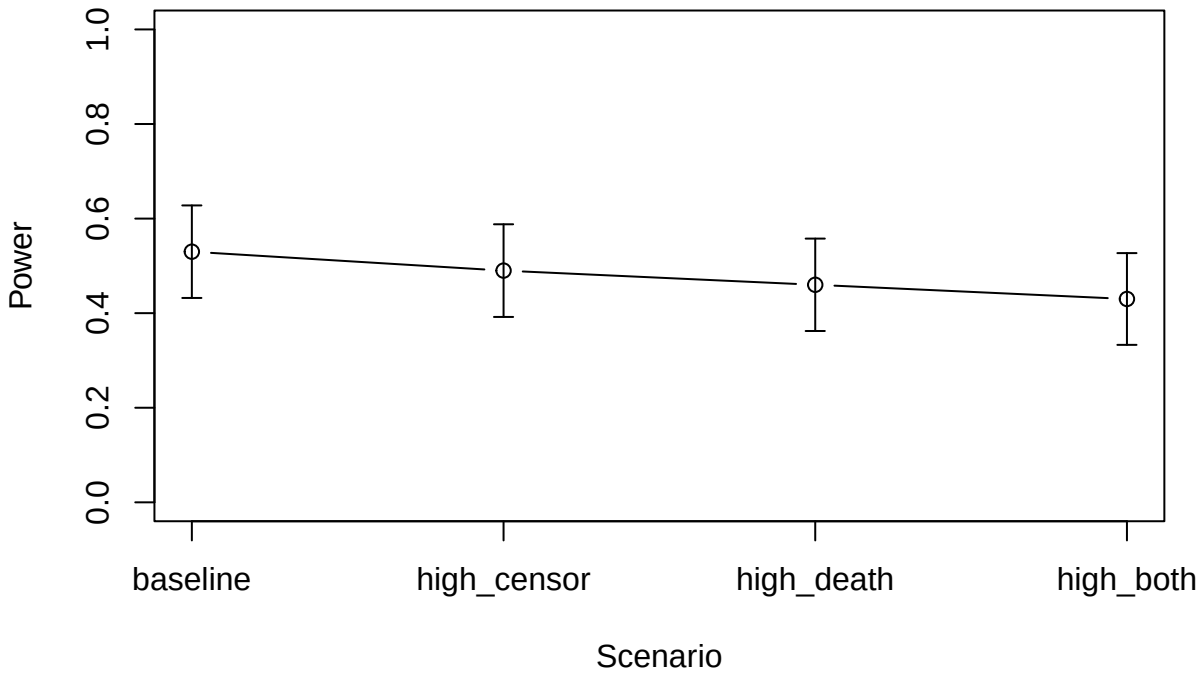
```
# Type II error

set.seed(123)

B <- 100
alpha <- 0.05
t_eval <- 2.5
alphaA <- log(1.2)

power_out <- data.frame(
  scenario = scenarios$scenario,
  lambda_c = scenarios$lambda_c,
  lambda_d = scenarios$lambda_d,
  power = NA_real_,
  mean_beta = NA_real_,
  se_power = NA_real_,
  lower = NA_real_,
  upper = NA_real_
)

for (j in seq_len(nrow(scenarios))) {
```

```r
  lambda_c <- scenarios$lambda_c[j]
  lambda_d <- scenarios$lambda_d[j]

  reject <- logical(B)
  beta <- numeric(B)

  for (b in 1:B) {

    source("01_generate_data.R")

    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)"] < alpha
  }

  power_out$power[j] <- mean(reject)
  power_out$mean_beta[j] <- mean(beta)

  power_out$se_power[j] <- sqrt(power_out$power[j] * (1 - power_out$power[j]) / B)
  power_out$lower[j] <- max(0, power_out$power[j] - 1.96 * power_out$se_power[j])
  power_out$upper[j] <- min(1, power_out$power[j] + 1.96 * power_out$se_power[j])
}

power_out
```

```
##      scenario lambda_c lambda_d power mean_beta   se_power     lower     upper
## 1    baseline     0.25     0.25  0.53 0.2033488 0.04990992 0.4321766 0.6278234
## 2 high_censor     0.40     0.25  0.49 0.2091464 0.04999000 0.3920196 0.5879804
## 3  high_death     0.25     0.40  0.46 0.1935348 0.04983974 0.3623141 0.5576859
## 4   high_both     0.40     0.40  0.43 0.1996379 0.04950758 0.3329652 0.5270348
```

```r
plot(seq_len(nrow(power_out)), power_out$power, type = "b",
     xaxt = "n",
     ylim = c(0, 1),
     xlab = "Scenario",
     ylab = "Power")

axis(1, at = seq_len(nrow(power_out)), labels = power_out$scenario)

arrows(
  x0 = seq_len(nrow(power_out)),
  y0 = power_out$lower,
  x1 = seq_len(nrow(power_out)),
  y1 = power_out$upper,
  angle = 90,
  code = 3,
  length = 0.05
```

```
)
```


Power
0.0
0.2
0.4
0.6
0.8
1.0
baseline
high_censor
high_death
high_both
Scenario